Stopwatch · Watch · Timer 미니프로젝트

타이머 시뮬레이션 검증

verification_v2 타이머 TB 3개 — 73개 검사 모두 PASS · 파형 24장 근거

정보윤 · 2026-10-17 · v1.0 (팀 보고용 — 검증 내용 설명)

Vivado 2022.2 behavioral 시뮬레이션 · Basys3 (xc7a35tcpg236-1) 소스 그대로 · tick 올리지 않은 실제 시간

개요 — 무엇을 검증했나

타이머 블록 (게이팅 → FSM → 데이터패스 → 화면 · LED) 을 TB 3개로 검증 — 21 + 41 + 11 = 73개 검사, fail 0

결과

TB	검증 내용	시뮬 시간	검사	결과
TM1	더하기 3종 · 초→분 올림 · 59:59 상한 · 0 에서 시작 금지	455 ms	21	**PASS**
TM2	실제 10초 카운트다운 · 알람 · LED 점멸 · 끄기 · 빌림	11,595 ms	41	**PASS**
TM3	RUN 중 버튼 무시 · 취소 · 백그라운드 · 화면 우선순위	555 ms	11	**PASS**

- **조건 —** verification_v2 (d413554) 소스 · TB 그대로, Vivado 2022.2 GUI
- **근거 —** 콘솔 [SUMMARY] ... => PASS 줄 + 구간별 파형 캡처
- **시간 —** clk 100 MHz · tick 10 ms 실제 값 (긴 구간만 프리로드)

- **TB · 안내서** 정안성 (Claude 공동 작성) **실행 · 캡처** 정보윤
- **상자 · 설명 · 슬라이드** 클로드코드로 작성, 정보윤 확인
- **타이머 RTL** 클로드코드 참고 구현 → 팀 v2 통합 (정안성)

목차

타이머 사양 · 신호 경로	**3–4**
검증 방법 (TB 구조 · 파형 보는 법)	**5**
TM1 시나리오 · 결과 11장	**6–17**
TM2 시나리오 · 결과 6장	**18–24**
TM3 시나리오 · 결과 7장	**25–32**
정리 · 팀이 알아야 할 것	**33**

타이머 사양 — 상태별 버튼 동작

SET · RUN · ALARM 3상태 (timer_control_unit.sv) — 더하기는 SET 에서만, RUN 은 L 만 받음, ALARM 은 아무 버튼이나 끄기

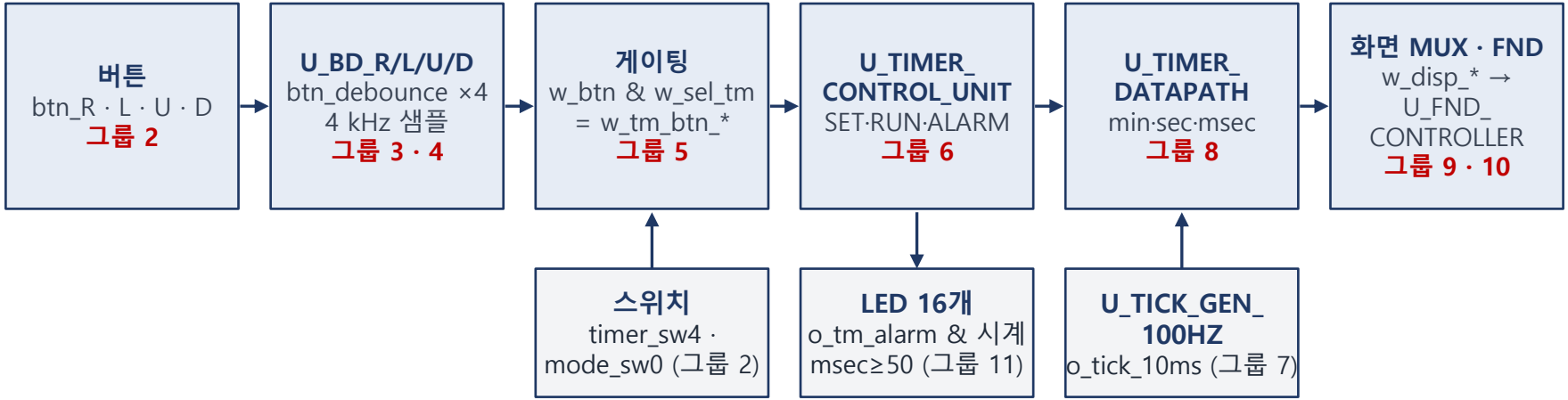
상태	U	R	D	L	그 밖
SET	+10초 (amount 0)	+1분 (amount 1)	+10분 (amount 2)	시작 → RUN (00:00 이면 무시)	최대 59:59 — 넘으면 59:59
RUN	무시	무시	무시	취소 → 00:00 · SET	10 ms 마다 −0.01초, 0 이면 ALARM
ALARM	끄기 → SET	끄기 → SET	끄기 → SET	끄기 → SET	LED 16개 0.5초 점멸, 값은 그대로

화면 · 버튼 · LED (stopwatch_watch.v)

- **화면 선택 —** timer_sw4 = 1 이면 타이머 화면 (mode_sw0 보다 우선). `w_sel_tm = timer_sw4`
- **버튼 게이팅 —** `w_tm_btn_* = w_btn_* & w_sel_tm` → 타이머 화면일 때만 타이머로 들어감 (다른 화면 버튼은 0)
- **백그라운드 —** 타이머 FSM · 데이터패스는 화면과 상관없이 계속 동작 → 시계 화면에서도 0 이 되면 ALARM
- **알람 LED —** `w_alarm_blink = (w_wt_msec >= 50)` → 시계 1/100초 기준 500 ms 켜짐 · 500 ms 꺼짐, 16개 모두 같이
- **10 ms tick —** U_TICK_GEN_100HZ 하나를 스톱워치 · 시계 · 타이머가 같이 씀 (항상 돌고, 타이머는 RUN 일 때만 받음)

신호 경로 — 파형 그룹 번호 = 블록 순서

버튼 하나가 디바운서 (2 ms) → 게이팅 → FSM 펄스 → 데이터패스 값 → 화면 · LED 로 가는 순서대로 파형 그룹 2 ~ 11 을 배치



- **그룹 1** clk · rst — 긴 TB2 는 clk 를 빼서 파형 파일 크기를 줄임 (그래도 11 GB)
- **파형 배치** script/timer_wave_v2.tcl 한 줄로 11개 그룹 · 색 · 진법 자동 (빨강 · 파랑은 X · Z 와 헷갈려서 안 씀)
- **캡처마다** 필요한 그룹만 펼치고, 볼 신호는 하늘색 강조, 근거는 빨간 상자 + 번호

검증 방법 — TB 구조 · 판정 · 콘솔

TB 는 Phase 마다 버튼을 누르고 (5 ms 누름 + 5 ms 뗌) 기대값과 비교 → PASS / FAIL 을 세서 마지막에 [SUMMARY] 한 줄

- **Phase → check —** Phase 하나 = 시나리오 한 덩어리. check 마다 실제 값과 기대값을 비교해 fail 수를 셈
- **프리로드 —** 59:59 처럼 버튼을 수십 번 눌러야 하는 값은 TB 가 데이터패스 레지스터에 직접 씀 (콘솔 [PRELOAD] 줄). TM2 P1 의 10초는 프리로드 없이 실제로 셈
- **판정 예 —** TM2 P1: L (시작) → 알람까지 간격이 (9,990 ms, 10,000 ms + 1클럭] 안이면 PASS
- **파형 캡처 —** 커서는 Go to next transition 으로 변화 순간에 맞춤 → 왼쪽 Value 칸이 그 순간 값. 10 ns 펄스는 ns 단위로 따로 확대

Vivado Tcl 콘솔 — 세 TB 마지막 줄 (vivado.log 에서 그대로 옮김)

```
[ 455.019060 ms] [SUMMARY] TM1 src=orig checks=21 fail=0 defect=0 limit=0 first_bad=- => PASS
[11595.002070 ms] [SUMMARY] TM2 src=orig checks=41 fail=0 defect=0 limit=0 first_bad=- => PASS
[ 555.009030 ms] [SUMMARY] TM3 src=orig checks=11 fail=0 defect=0 limit=0 first_bad=- => PASS
```

src=orig = 원본 소스 그대로 (bug 주입 없음) · defect = 사양과 다른 동작 · limit = 알려진 한계 — 세 TB 모두 0

TM1 시나리오 — 더하기 · 상한 (455 ms)

Phase 7개 · 검사 21개 모두 PASS — 더하기 · 올림 · 59:59 상한을 버튼 입력으로 확인 (P6 만 프리로드)

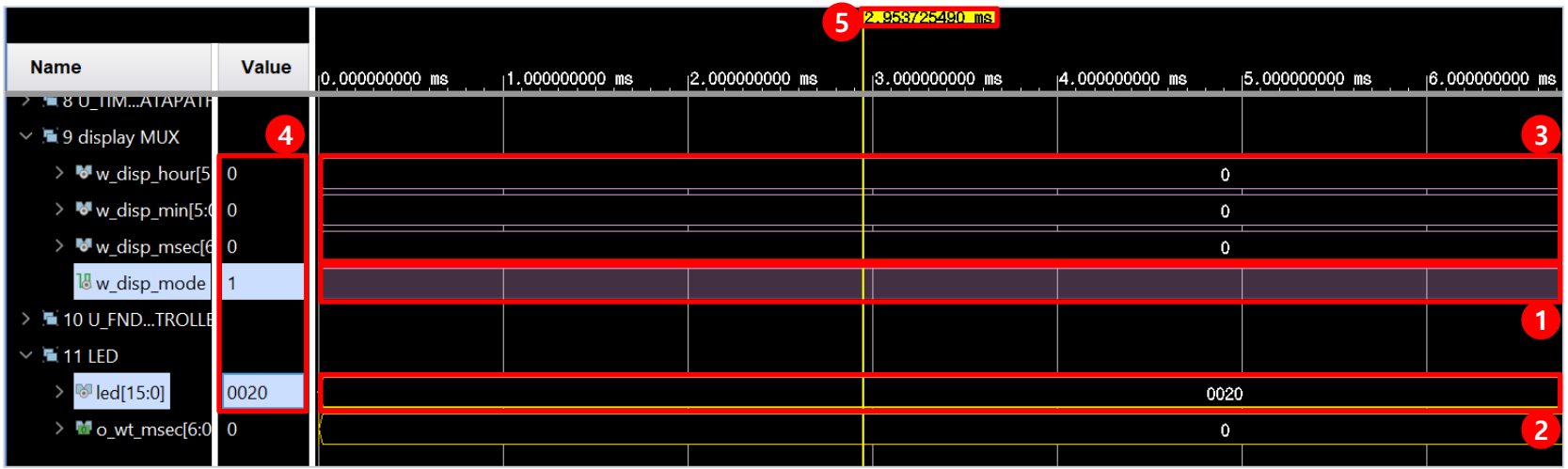
Phase	시각 (ms)	TB 동작	기대 결과 (판정)	검사	캡처
P1	0 – 5	리셋 직후 (타이머 화면)	disp_mode 1 · LED 0020 · FND 00.00	3	7
P2	5 – 15	00:00 에서 L	시작 안 함 (SET · run 0)	1	8
P3	15 – 80	U ×6	초 10 … 50 → 01:00 (초→분 올림)	3	9–12
P4	80 – 110	R · D · U	01:00 → 11:00 → 11:10	3	13
P5	110 – 315	D ×5 · R ×9 · U ×5 · U	50:00 → 59:00 → 59:50 → 59:59	5	14
P6	315 – 355	59:59 프리로드 후 U · R · D	59:59 그대로	3	15
P7	355 – 455	52:10 에서 D (+10분)	62:10 이 아니라 59:59	3	16

```
# Vivado Tcl 콘솔 — TM1 (vivado.log, 줄 끝 | disp=… 생략)
[   5.003010 ms] [ PASS] [TM1.P1.-] P1 checks=3 all passed (defect=0)
[  15.006010 ms] [ PASS] [TM1.P2.-] P2 checks=1 all passed (defect=0)
[  80.009010 ms] [ PASS] [TM1.P3.-] P3 checks=3 all passed (defect=0)
[ 110.012010 ms] [ PASS] [TM1.P4.-] P4 checks=3 all passed (defect=0)
[ 315.015010 ms] [ PASS] [TM1.P5.-] P5 checks=5 all passed (defect=0)
[ 325.016050 ms] [ INFO] [TM1.P6.PRELOAD] dut.U_TIMER_DATAPATH.{min,sec,msec}_reg = 59:59.00
[ 355.016060 ms] [ PASS] [TM1.P6.-] P6 checks=3 all passed (defect=0)
[ 455.019060 ms] [ PASS] [TM1.P7.-] P7 checks=3 all passed (defect=0)
[ 455.019060 ms] [SUMMARY] TM1 src=orig checks=21 fail=0 defect=0 limit=0 first_bad=- => PASS
```

TM1 ① P1 — 리셋 직후 타이머 화면

리셋 뒤 timer_sw4 = 1 이면 바로 타이머 화면: w_disp_mode 1 (분.초 표시) · LED 0020 (led5) · 값 00:00.00

TM1 · 0 ~ 6 ms · 커서 2.958 ms · 그룹 9 · 11

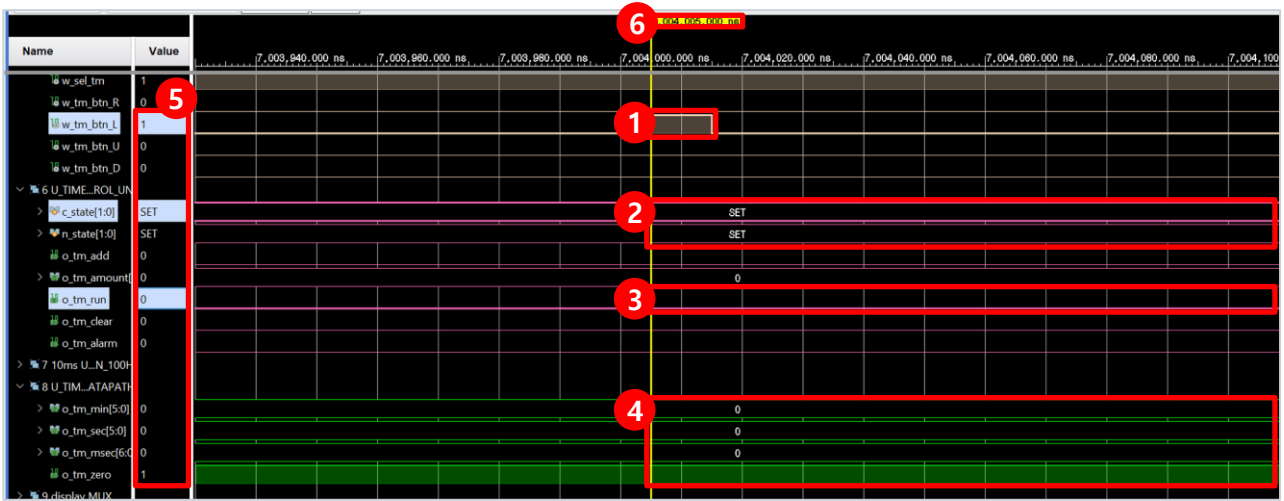


① **w_disp_mode 1** = 타이머 화면 (분.초)
② **LED 0020** = led5 하나 (타이머 화면 표시등)
③ 화면 값 0 · 0 · 0 → FND 00.00
④ 커서 Value: 0 · 0 · 0 · 1 · 0020
⑤ 커서 시각 2.958 ms

TM1 ② P2 — 00:00 에서 L 은 시작 안 함

L 펄스는 타이머까지 들어오지만 o_tm_zero = 1 이라 SET 그대로 — 남은 시간이 0 이 아닐 때만 시작 (소스 주석 그대로)

TM1 · 7,003,930 ~ 7,004,100 ns (ns 확대) · 커서 7,004,005 ns · 그룹 5 · 6 · 8



① **w_tm_btn_L** 10 ns 펄스 (게이팅 통과)

② 상태는 **SET 그대로** (RUN 으로 안 감)

③ **o_tm_run 0** 그대로

④ 값 00:00.00 · **o_tm_zero 1** — timer_control_unit.sv 46행 if (i_tm_zero == 1'b0) n_state = RUN

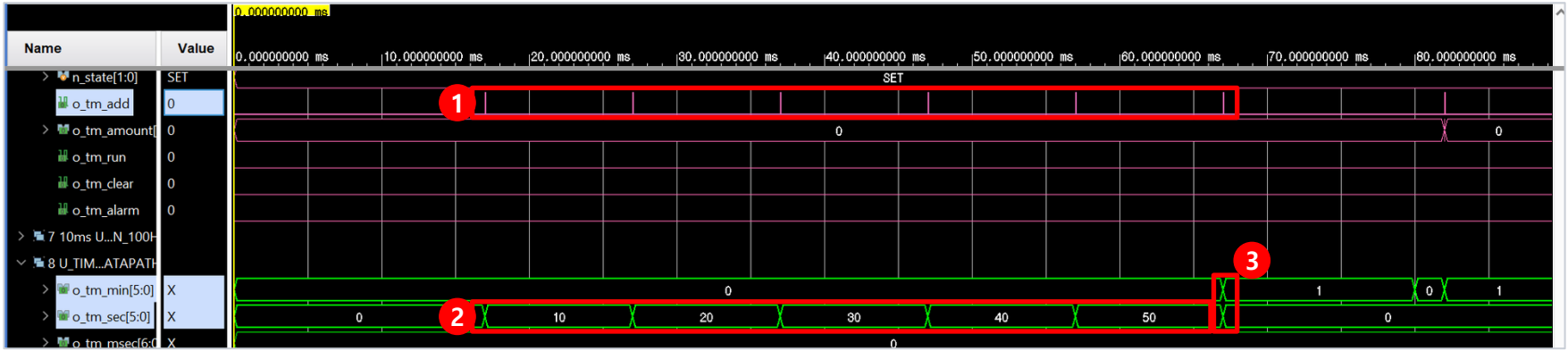
⑤ 커서 Value: L 1 · SET · run 0 · zero 1

⑥ 커서 시각 7,004,005 ns

TM1 ③ P3a — U 6번 → 50초 다음은 01:00

U 한 번에 +10초 (o_tm_add 펄스 6개) — 초 10 → 20 → 30 → 40 → 50, 6번째는 60초가 아니라 01:00 으로 올림

TM1 · 0 ~ 88 ms · 그룹 6 · 8 (커서 0 ms 라 Value 칸은 X — 파형만 볼 것)



① U 6번 → **o_tm_add** 펄스 6개 (SET 상태)

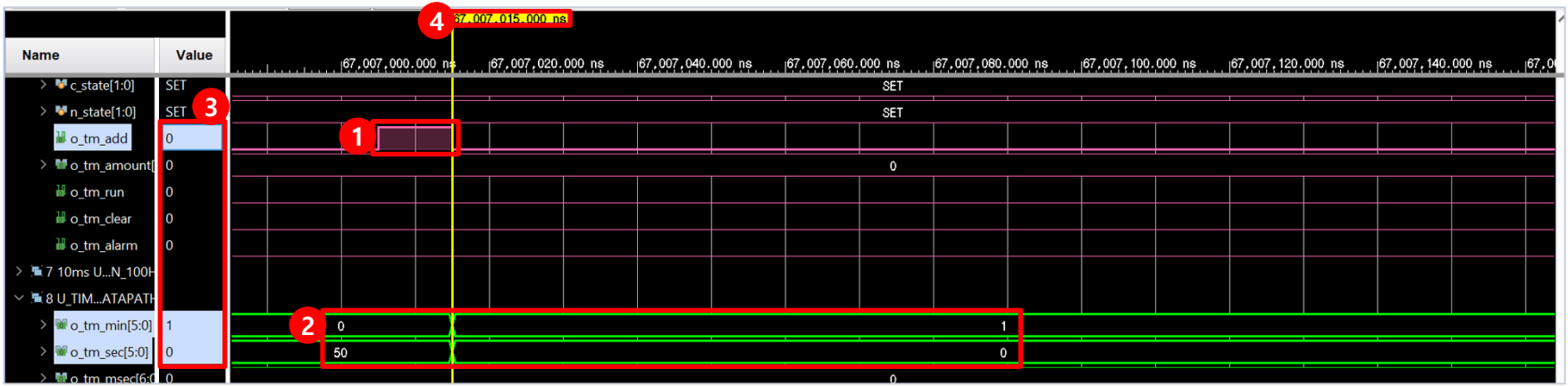
② 초 10 → 20 → 30 → 40 → 50

③ 6번째: 00:50 → **01:00** (확대는 다음 장)

TM1 ④ P3b — 올림 순간 확대

o_tm_add 펄스 다음 클럭에 분 0 → 1 · 초 50 → 0 이 동시에 — 데이터패스가 sec + 10 ≥ 60 이면 −60 하고 분에 carry

TM1 · 67,006,990 ~ 67,007,160 ns · 커서 67,007,015 ns · 그룹 6 · 8



① 6번째 U → **o_tm_add** 펄스 (+10초 요청)

② 다음 클럭: **00:50 → 01:00**

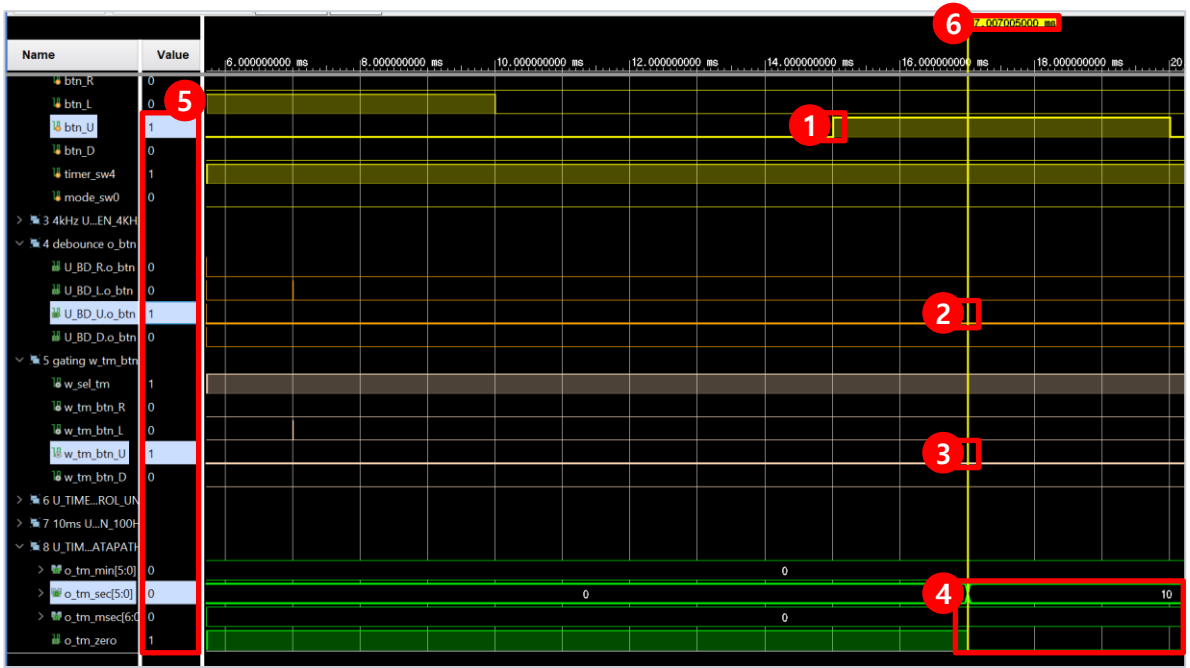
③ 커서 Value: add 0 · min 1 · sec 0

④ 커서 시각 67,007,015 ns

TM1 ⑤ P3c — 버튼 경로 (ms): 누름 → 2 ms 뒤 인식

btn_U 를 15.0 ms 에 누름 → 디바운서가 4 kHz 로 샘플해 17.007 ms 에 1클럭 펄스 → 같은 순간 게이팅 통과 → 초 +10

TM1 · 5 ~ 20 ms · 커서 17.007005 ms · 그룹 2 · 4 · 5 · 8



① **btn_U** 누름 (15.0 ms)

② **U_BD_U.o_btn** 펄스 17.007 ms — 약 2 ms 뒤 인식

③ 같은 순간 **w_tm_btn_U** (timer_sw4 = 1)

④ 초 0 → 10 · o_tm_zero 1 → 0

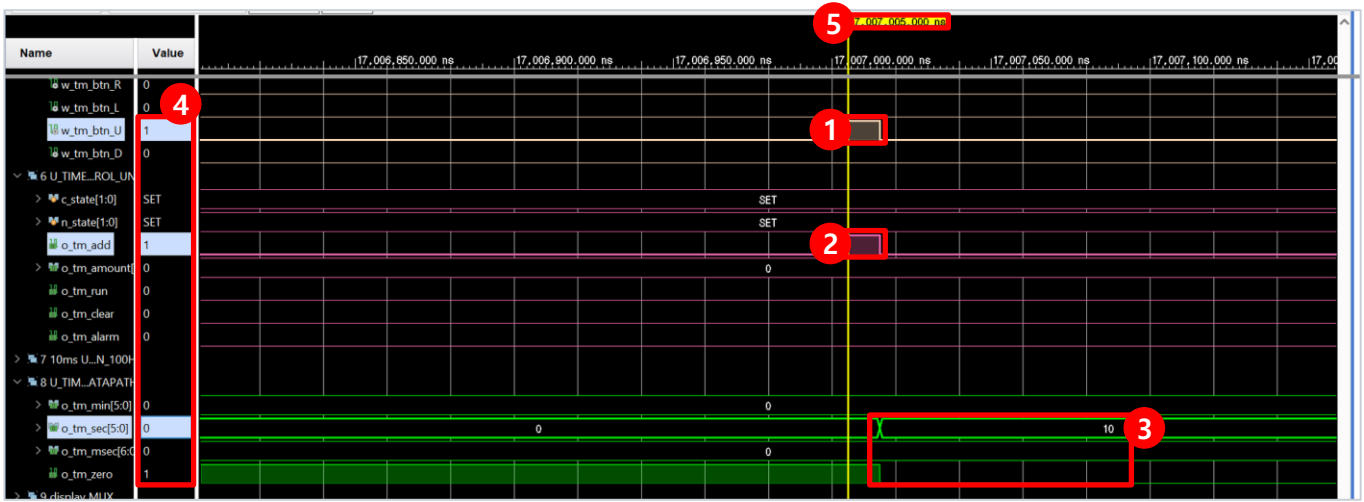
⑤ 커서 Value: btn_U 1 · U_BD_U 1 · w_tm_btn_U 1 · sec 0 (다음 클럭 10)

⑥ 커서 시각 17.007005 ms

TM1 ⑥ P3c — 버튼 경로 (ns): 같은 클럭 · 다음 클럭

w_tm_btn_U 와 같은 클럭에 o_tm_add 1 (밀리 출력) → 다음 상승 에지 (10 ns 뒤) 에 레지스터가 바뀌어 초 0 → 10

TM1 · 17,006,810 ~ 17,007,170 ns · 커서 17,007,005 ns · 그룹 5 · 6 · 8

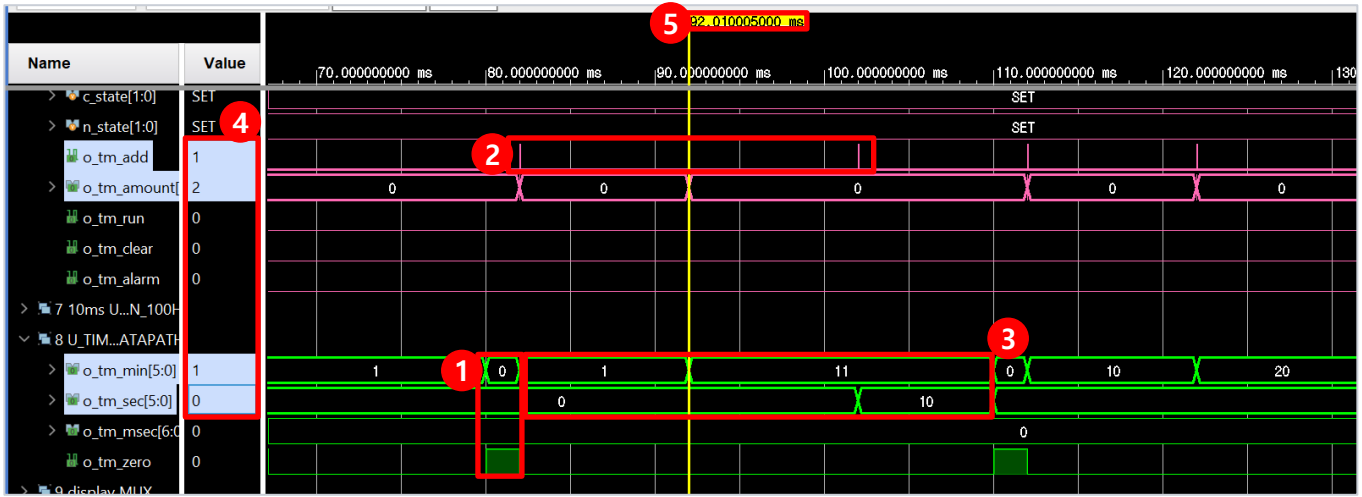


① **w_tm_btn_U** 10 ns 펄스
② 같은 클럭 **o_tm_add 1** (SET 상태 조합 출력)
③ 다음 클럭: 초 0 → 10, o_tm_zero 0
④ 커서 Value: w_tm_btn_U 1 · add 1 · sec 0
⑤ 커서 시각 17,007,005 ns

TM1 ⑦ P4 — R · D · U = +1분 · +10분 · +10초

o_tm_amount 로 더할 양을 구분 (0 = +10초, 1 = +1분, 2 = +10분) — 01:00 → 11:00 → 11:10

TM1 · 64 ~ 132 ms · 커서 92.010005 ms (D 펄스) · 그룹 6 · 8



① 80 ms TB 리셋 → 00:00 (Phase 새로 시작)

② **o_tm_add** 3번: R (amount 1) · D (2) · U (0)

③ 01:00 → **11:00** → **11:10**

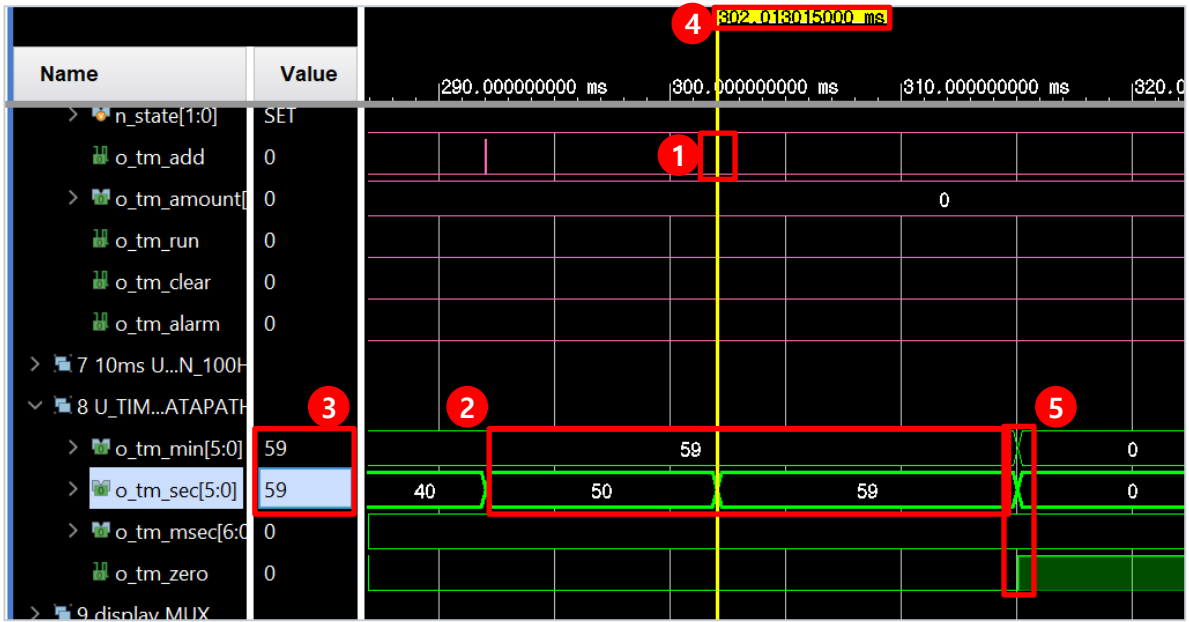
④ 커서 Value: **amount 2** = D +10분 (timer_control_unit.sv 57행), min 은 다음 클럭에 11

⑤ 커서 시각 92.010005 ms

TM1 ⑧ P5 — 59:50 + 10초 → 59:59 에서 멈춤

분.초 4자리 화면이라 60분 이상은 표시 불가 → 더한 결과가 60분 이상이면 59:59 로 고정 (timer_datapath.vw_over)

TM1 · 285 ~ 320 ms · 커서 302.013015 ms · 그룹 6 · 8



① 마지막 U (펄스는 커서 선 아래)

② 59:50 + 10초 → **59:59** (60:00 아님)

③ 커서 Value: min 59 · sec 59

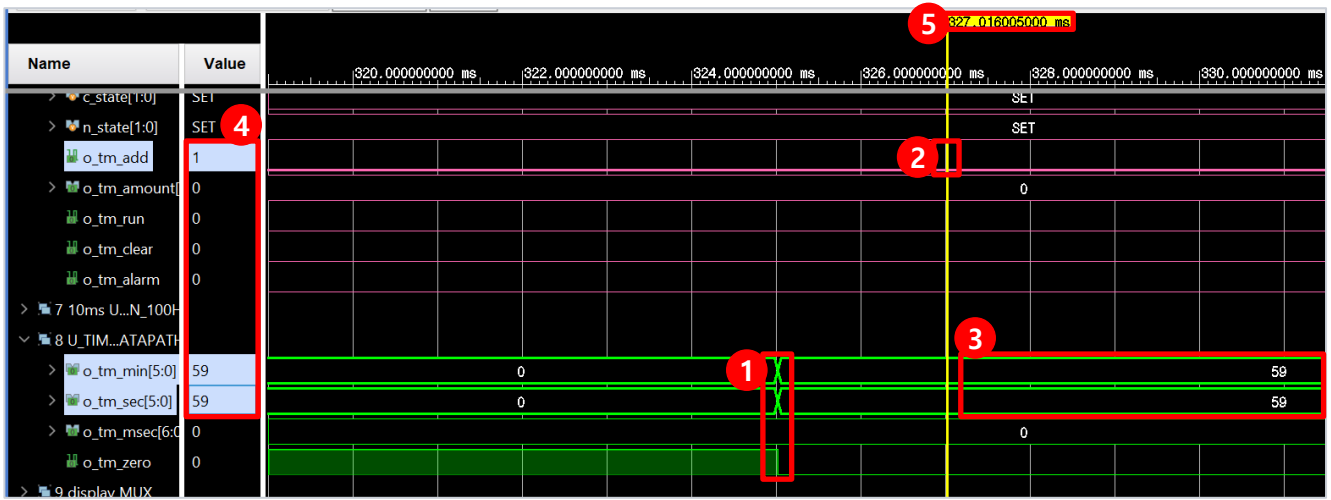
④ 커서 시각 302.013015 ms

⑤ 313 ms TB 리셋 → 00:00 (다음 Phase)

TM1 ⑨ P6 — 59:59 에서 더해도 그대로

TB 가 325 ms 에 59:59 를 써 넣은 뒤 U · R · D — o_tm_add 펄스는 나오지만 값은 59:59 유지

TM1 · 318 ~ 331 ms · 커서 327.016005 ms · 그룹 6 · 8

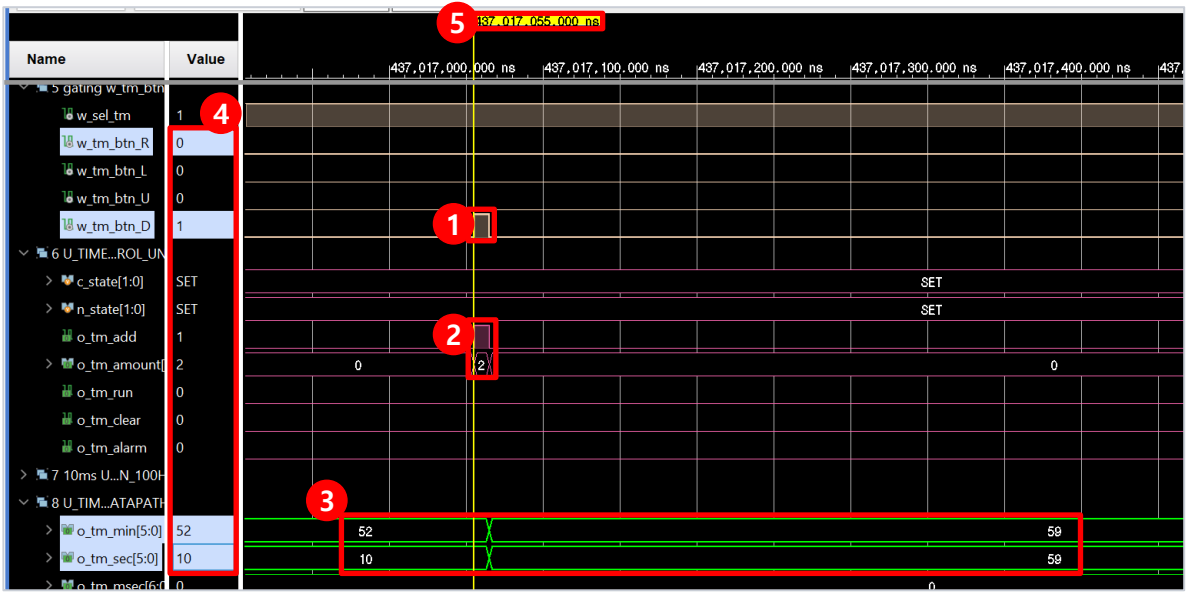


① 325 ms 프리로드 59:59 (콘솔 [PRELOAD] 줄)
② U → **o_tm_add** 펄스 (+10초 요청)
③ 그래도 **59:59 그대로**
④ 커서 Value: add 1 · amount 0 · 59 · 59
⑤ 커서 시각 327.016005 ms

TM1 ⑩ P7 — 52:10 + 10분 → 59:59

62:10 이 될 덧셈도 59:59 에서 멈춤 — 분 단위 덧셈 (D) 에서도 상한이 동작

TM1 · 437,016,960 ~ 437,017,500 ns · 커서 437,017,055 ns · 그룹 5 · 6 · 8



① **w_tm_btn_D** 10 ns 펄스

② **o_tm_add 1 · amount 2** (+10분)

③ 52:10 → **59:59** (62:10 아님)

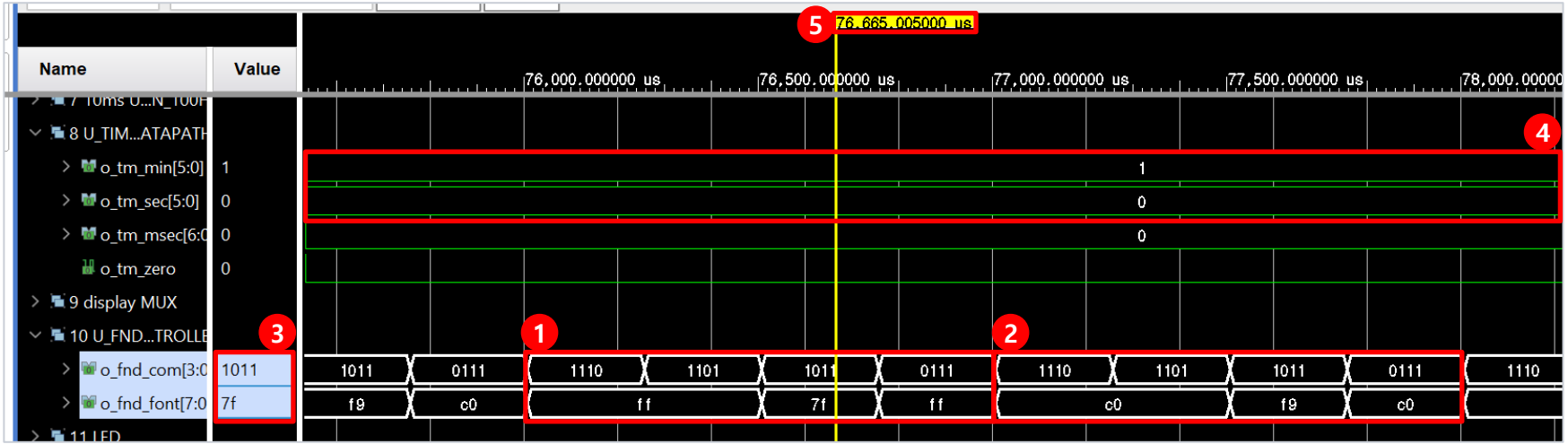
④ 커서 Value: D 1 · add 1 · amount 2 · 52 · 10

⑤ 커서 시각 437,017,055 ns

TM1 ⑪ FND — 01:00 이 "01.00" 으로 스캔

한 바퀴 = 8칸 × 250 µs (2 ms): 숫자 4칸 + 빈칸 4칸, 빈칸 중 1011 칸에서 가운데 점 (7f) — SW2_FND_CONTROLLER.v mux_8x1

TM1 · 75.75 ~ 78.2 ms (80 ms 리셋 전) · 커서 76,665.005 µs · 그룹 8 · 10



① 빈칸 4칸: ff · ff · **7f (가운데 점)** · ff

② 숫자 4칸: 1110 c0 (0) · 1101 c0 (0) · 1011 f9 (1) · 0111 c0 (0) → "0100"

③ 커서 Value: com 1011 · font 7f (점 칸)

④ 타이머 값 01:00 (min 1 · sec 0)

⑤ 커서 시각 76,665.005 µs — 점은 1/100초 < 50 일 때만 켬 (dot.v 11행)

TM2 시나리오 — 카운트다운 · 알람 (11,595 ms)

Phase 4개 · 검사 41개 모두 PASS — 10초는 프리로드 없이 실제로 세고, 알람 LED 는 1.5초 동안 30번 샘플

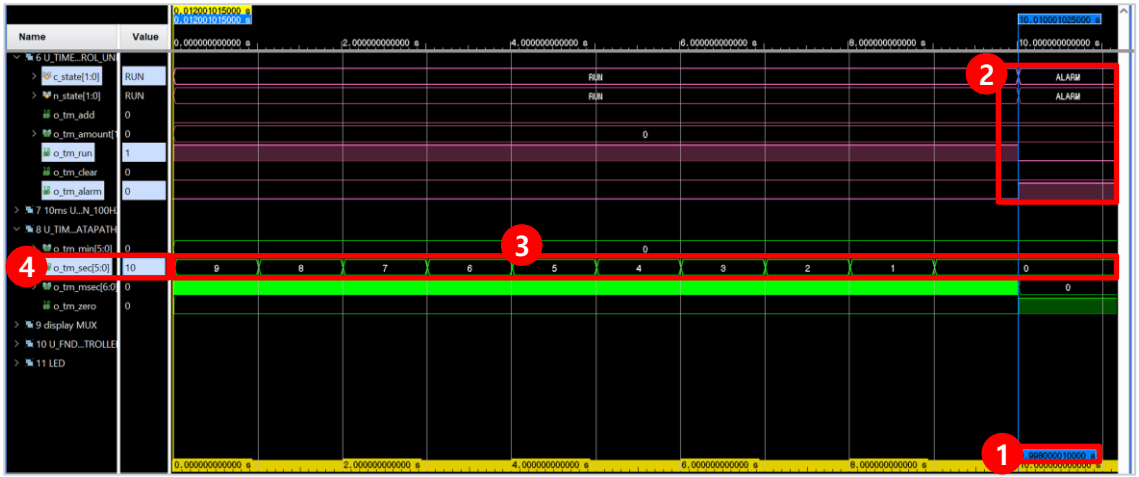
Phase	시각 (ms)	TB 동작	기대 결과 (판정)	검사	캡처
P1	0 – 10,010	00:10 설정 → L (시작)	run → alarm 간격 (9,990, 10,000 ms + 1클럭]	4	19
P2	10,010 – 11,510	알람 1.5초 그대로 둠	LED 16개 = 시계 msec ≥ 50 이면 ffff	31	20–22
P3	11,510 – 11,520	알람 중 D	알람 끔 → SET, 값 0 그대로 (+10분 아님)	3	23
P4	11,520 – 11,595	프리로드 01:00.00 · 10:00.00	00:59.99 · 09:59.99 (분에서 빌림)	3	24

```
# Vivado Tcl 콘솔 — TM2 (vivado.log, 줄 끝 | disp=… 생략)
[10010.001025 ms] [ INFO] [TM2.P1.TB5] run->alarm = 9998.000010 ms (set 10.000 s)
[10010.001025 ms] [ PASS] [TM2.P1.-] P1 checks=4 all passed (defect=0)
[11510.001025 ms] [ INFO] [TM2.P2.TC6] 30 samples over 1.5 s, 0 mismatches
[11510.001025 ms] [ PASS] [TM2.P2.-] P2 checks=31 all passed (defect=0)
[11520.001030 ms] [ PASS] [TM2.P3.-] P3 checks=3 all passed (defect=0)
[11550.002070 ms] [ INFO] [TM2.P4.PRELOAD] dut.U_TIMER_DATAPATH.{min,sec,msec}_reg = 01:00.00
[11570.002070 ms] [ INFO] [TM2.P4.PRELOAD] dut.U_TIMER_DATAPATH.{min,sec,msec}_reg = 10:00.00
[11595.002070 ms] [ PASS] [TM2.P4.-] P4 checks=3 all passed (defect=0)
[11595.002070 ms] [SUMMARY] TM2 src=orig checks=41 fail=0 defect=0 limit=0 first_bad=- => PASS
```

TM2 ① P1 — 10초 설정 → 9.998 s 뒤 알람

o_tm_run↑ (12.001 ms, L 인식) → o_tm_alarm↑ (10,010.001 ms) 간격 9,998.000010 ms — TB 합격 범위 (9,990, 10,000 + 1클럭] 안

TM2 · 0 ~ 10.02 s · 마커 12.001015 ms (o_tm_run↑) · 커서 10.010001 s (o_tm_alarm↑) · 그룹 6 · 8



① 마커 → 커서 간격 **9.998000010 s** (아래 Floating Ruler)

② **RUN → ALARM**, o_tm_run 0 · o_tm_alarm 1

③ 초 9 → 8 → … → 1 → 0 (1초마다)

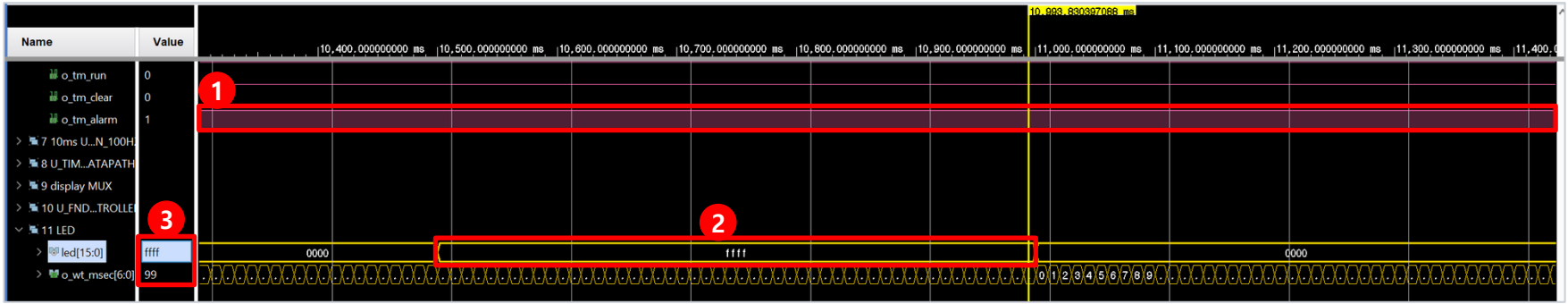
④ 마커 순간 Value: sec 10 (설정값)

⑤ **왜 2 ms 짧나**: 10 ms tick 을 공유해서 시작 시점이 tick 사이 어디든 첫 감소가 8 ms 뒤에 옴 → 8 + 999 × 10 = 9,998 ms

TM2 ② P2 — 알람 LED 500 ms 점멸 (전체)

ALARM 동안 LED 16개가 시계 1/100초 ≥ 50 이면 ffff, 아니면 0000 — 10,500 ms 켜짐 · 11,000 ms 꺼짐

TM2 · 10,300 ~ 11,450 ms · 커서 10,993.88 ms · 그룹 6 · 11



① **o_tm_alarm** 1 유지 (ALARM)

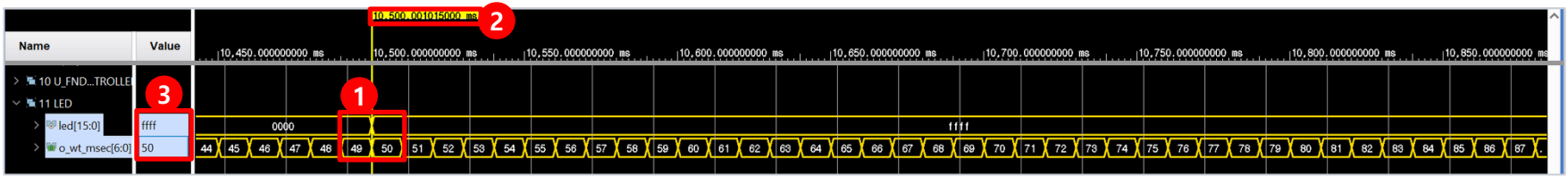
② **led ffff** 구간 = 시계 1/100초 50 ~ 99 (500 ms)

③ 커서 Value: ffff · 시계 msec 99

TM2 ③ P2 — 켜지는 순간 (49 → 50)

시계 1/100초가 49 → 50 이 되는 클럭에 led 0000 → ffff — stopwatch_watch.v 270행 w_alarm_blink = (w_wt_msec >= 50)

TM2 · 10,440 ~ 10,890 ms · 커서 10,500.001015 ms · 그룹 11



① 시계 msec **49 → 50** 과 같은 순간 led **0000 → ffff**

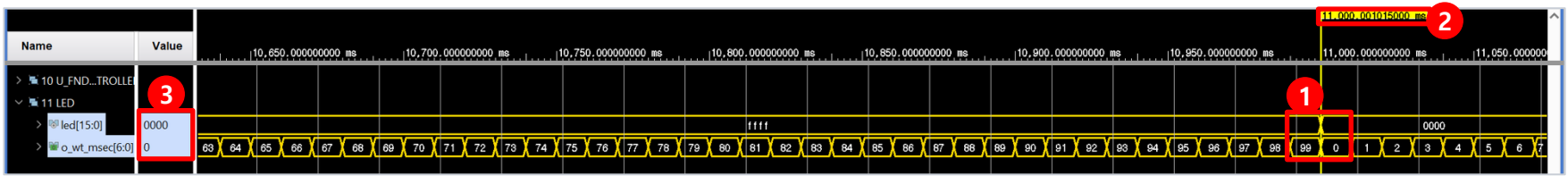
② 커서 시각 10,500.001015 ms

③ 커서 Value: ffff · 50

TM2 ④ P2 — 꺼지는 순간 (99 → 0)

시계 1/100초가 99 → 0 (다음 초) 이 되는 클럭에 led ffff → 0000 — 켜짐 500 ms · 꺼짐 500 ms 반복

TM2 · 10,640 ~ 11,090 ms · 커서 11,000.001015 ms · 그룹 11



① 시계 msec **99 → 0** 과 같은 순간 led **ffff → 0000**

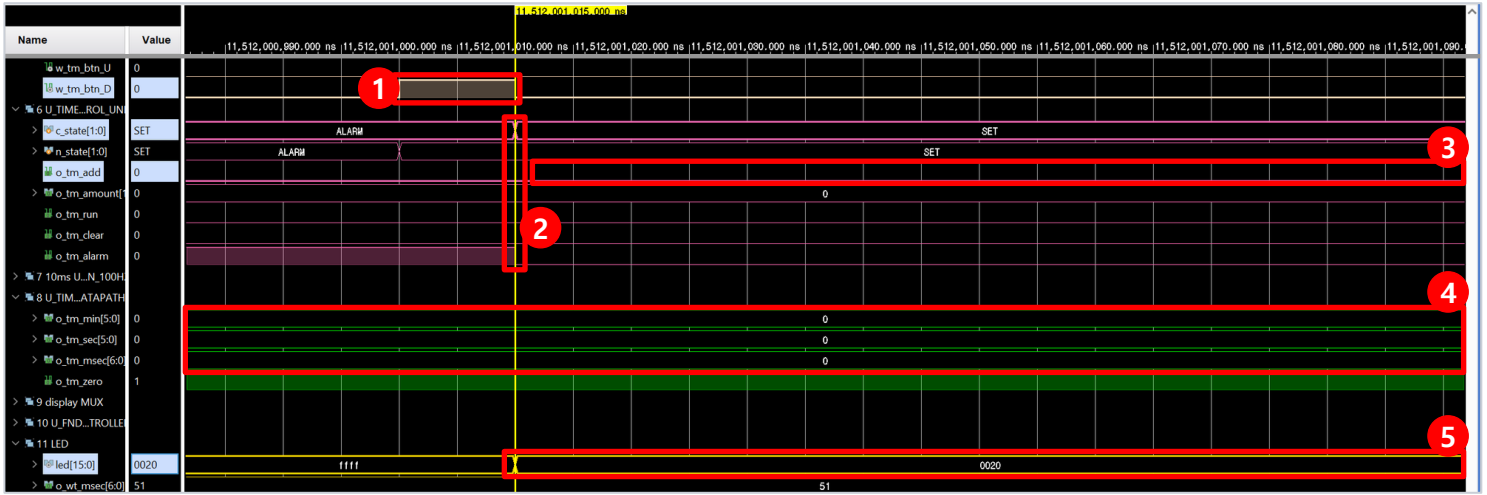
② 커서 시각 11,000.001015 ms

③ 커서 Value: 0000 · 0

TM2 ⑤ P3 — 알람 중 D → 끄기만 함

ALARM 에서는 D 가 +10분이 아니라 끄기 — 다음 클럭에 SET, o_tm_add 안 나옴, 값 00:00.00, LED ffff → 0020

TM2 · 11,512,000,985 ~ 11,512,001,095 ns · 커서 11,512,001,015 ns · 그룹 5 · 6 · 8 · 11



① **w_tm_btn_D** 10 ns 펄스

② n_state 는 펄스와 동시에 SET, **c_state 는 다음 클럭** (11,512,001,015 ns) 에 SET · alarm 0

③ **o_tm_add 0** — 더하지 않음

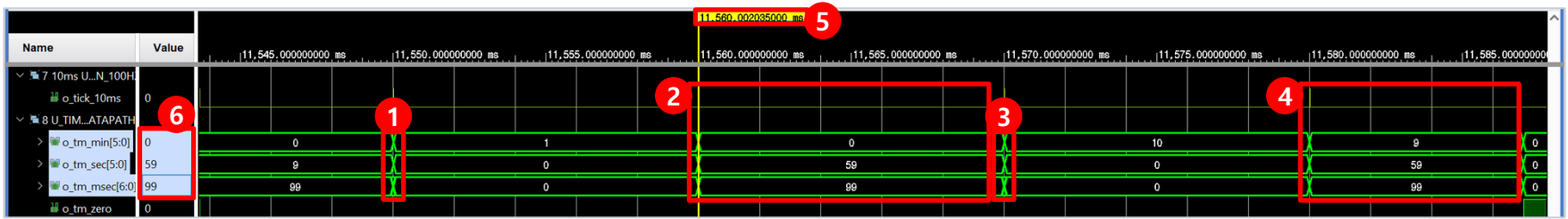
④ 값 00:00.00 그대로

⑤ LED **ffff → 0020** (알람 끝 → 타이머 화면 표시등)

TM2 ⑥ P4 — 카운트다운 빌림

01:00.00 → 다음 tick 에 00:59.99, 10:00.00 → 09:59.99 — msec 0 이면 99 로, 윗자리에서 1 빌림 (timer_datapath.v)

TM2 · 11,542 ~ 11,587 ms · 커서 11,560.002035 ms · 그룹 6 · 7 · 8



① 11,550 ms 프리로드 01:00.00 (tick 직후 50 ns 에 덮어씀)

② 다음 tick → **01:00.00 → 00:59.99**

③ 11,570 ms 프리로드 10:00.00

④ 다음 tick → **10:00.00 → 09:59.99**

⑤ 커서 시각 11,560.002035 ms

⑥ 커서 Value: 0 · 59 · 99

TM3 시나리오 — 취소 · 백그라운드 · 우선순위 (555 ms)

Phase 5개 · 검사 11개 모두 PASS — 시계 화면에서도 타이머가 돌아 알람이 울리고, 끄는 것은 타이머 화면 버튼만

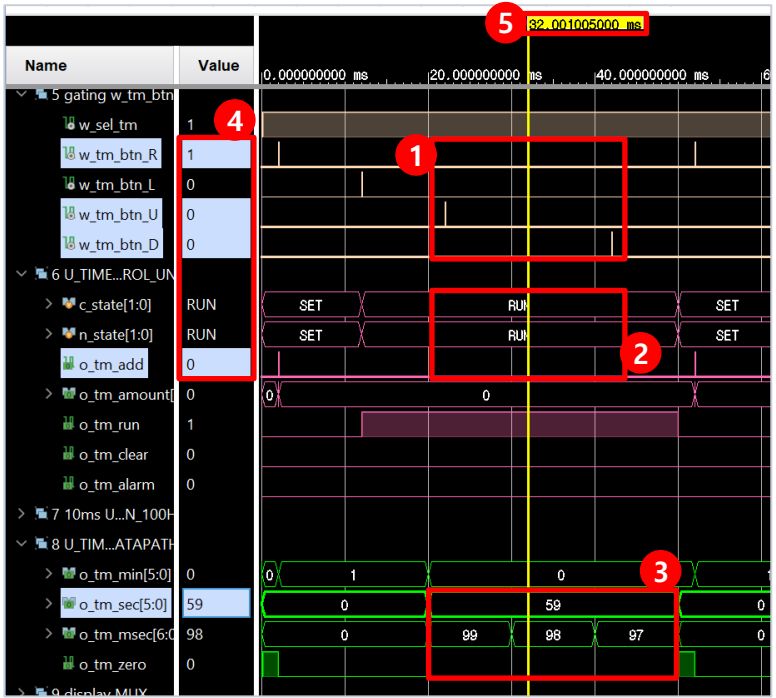
Phase	시각 (ms)	TB 동작	기대 결과 (판정)	검사	캡처
P1	0 – 50	RUN 중 U · R · D	무시 (값은 줄기만)	2	26–27
P2	50 – 130	RUN 중 L	취소 → 00:00 · SET	1	28–29
P3	130 – 485	시작 → 시계 화면 → 프리로드 00:00.30	시계 화면인데 460 ms ALARM · LED 패턴	3	30
P4	485 – 545	시계 · 스톱워치 화면 버튼	알람 유지 (못 끔)	2	31
P5	545 – 555	sw4 = 1 · sw0 = 1 → R	타이머 화면 우선, 알람 끔, +1분 안 함	3	32

```
# Vivado Tcl 콘솔 — TM3 (vivado.log, 줄 끝 | disp=… 생략)
[  50.003010 ms] [ PASS] [TM3.P1.-] P1 checks=2 all passed (defect=0)
[ 130.006010 ms] [ PASS] [TM3.P2.-] P2 checks=1 all passed (defect=0)
[ 160.007050 ms] [ INFO] [TM3.P3.PRELOAD] dut.U_TIMER_DATAPATH.{min,sec,msec}_reg = 00:00.30
[ 460.007025 ms] [ INFO] [TM3.P3.TC3] ALARM reached while watch screen is shown
[ 485.007025 ms] [ PASS] [TM3.P3.-] P3 checks=3 all passed (defect=0)
[ 545.008030 ms] [ PASS] [TM3.P4.-] P4 checks=2 all passed (defect=0)
[ 555.009030 ms] [ PASS] [TM3.P5.-] P5 checks=3 all passed (defect=0)
[ 555.009030 ms] [SUMMARY] TM3 src=orig checks=11 fail=0 defect=0 limit=0 first_bad=- => PASS
```

TM3 ① P1 — RUN 중 U · R · D 무시 (전체)

RUN 상태 FSM 은 L 만 검사 → U (22 ms) · R (32 ms) · D (42 ms) 펄스가 와도 o_tm_add 없음, 값은 줄기만

TM3 · 0 ~ 61 ms · 커서 32.001005 ms (R 펄스) · 그룹 5 · 6 · 8

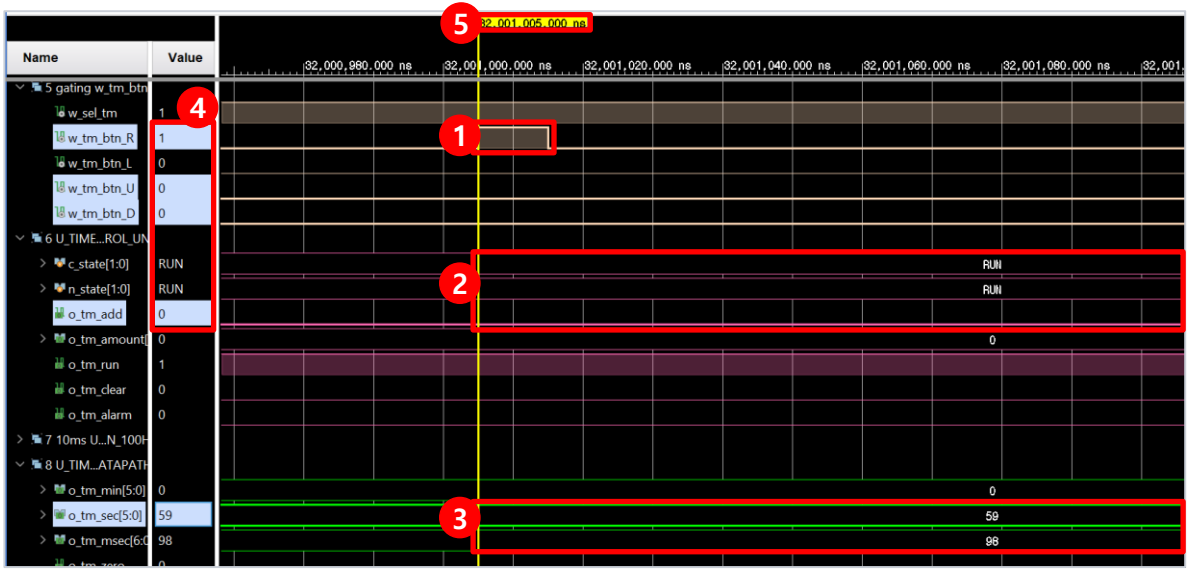


① RUN 중 U · R · D 입력 (R 은 커서 선에 가림)
② 상태 **RUN 그대로**, o_tm_add 펄스 없음
③ 값은 줄기만: 00:59.99 → 59.98 → 59.97
④ 커서 Value: R 1 · RUN · add 0
⑤ 커서 시각 32.001005 ms

TM3 ② P1 — R 펄스 확대

R 10 ns 펄스가 타이머까지 들어왔는데도 RUN 그대로 · o_tm_add 0 · 값 00:59.98 유지 → 1분 안 더해짐

TM3 · 32,000,960 ~ 32,001,130 ns · 커서 32,001,005 ns · 그룹 5 · 6 · 8



① **w_tm_btn_R** 10 ns 펄스 (게이팅 통과)

② 펄스 뒤에도 RUN, **o_tm_add 0**

③ 값 00:59.98 유지

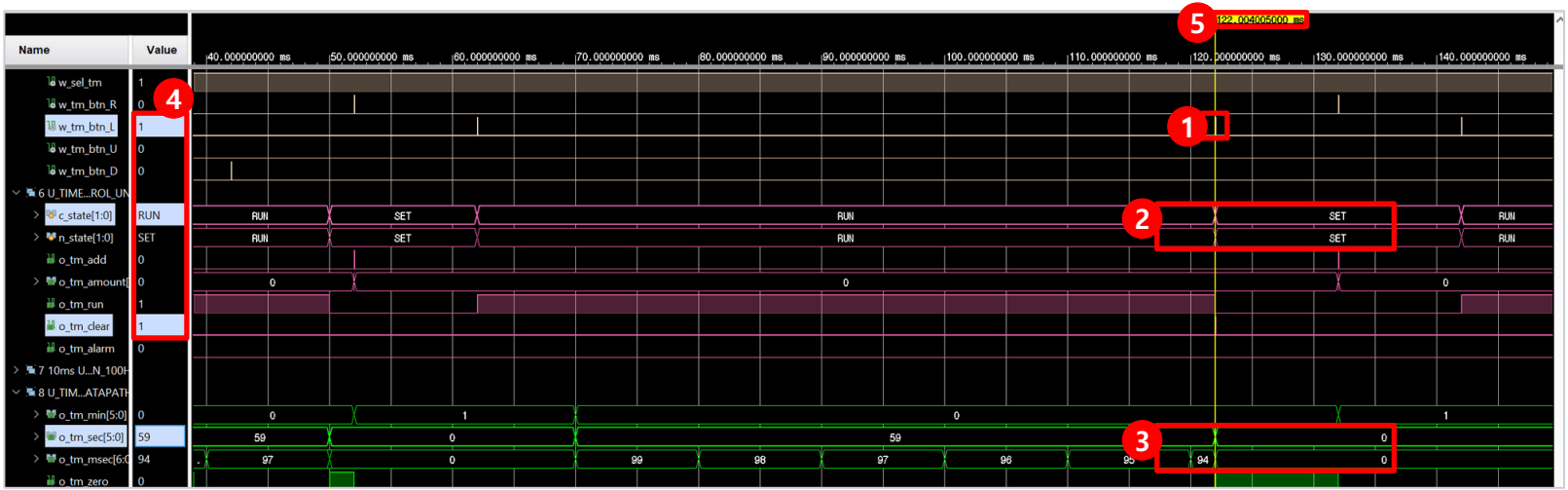
④ 커서 Value: R 1 · RUN · add 0

⑤ 커서 시각 32,001,005 ns

TM3 ③ P2 — RUN 중 L → 취소 (전체)

62 ms L = 시작, 122 ms L = 취소 — 같은 버튼이 SET 에서는 시작, RUN 에서는 취소

TM3 · 38 ~ 150 ms · 커서 122.004005 ms · 그룹 5 · 6 · 8



① RUN 중 L 펄스 (122 ms)

② 상태 RUN → **SET**

③ 값 00:59.94 → **00:00.00**

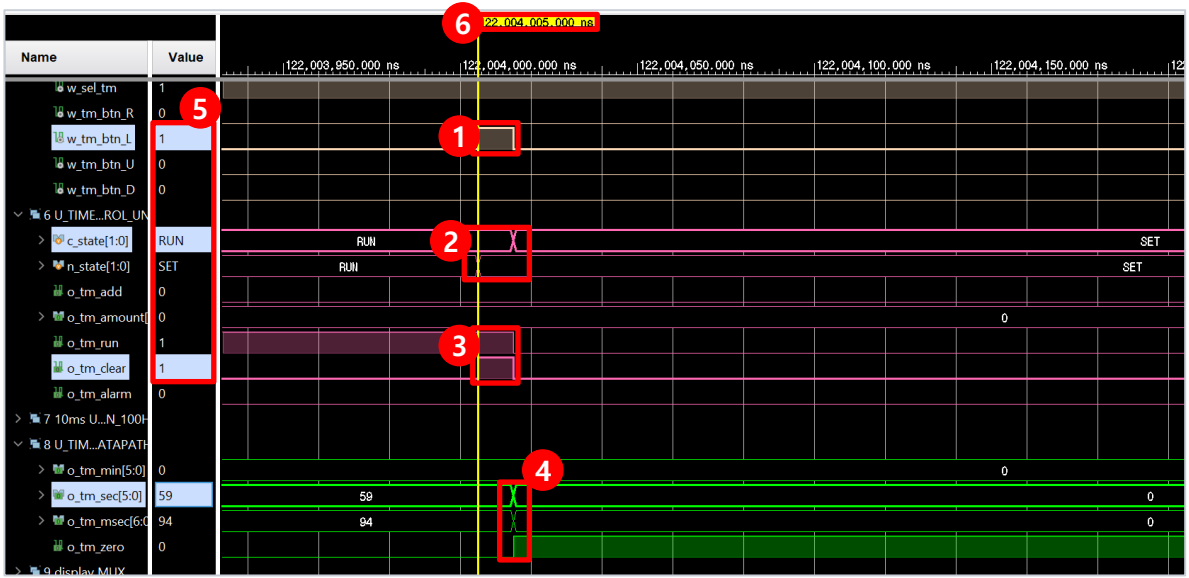
④ 커서 Value: L 1 · o_tm_clear 1

⑤ 커서 시각 122.004005 ms

TM3 ④ P2 — 취소 순간 확대

L 펄스 동안 o_tm_clear 1 (조합 출력) → 다음 클럭에 데이터패스 0 · c_state SET · o_tm_run 0 이 한꺼번에

TM3 · 122,003,930 ~ 122,004,170 ns · 커서 122,004,005 ns · 그룹 5 · 6 · 8



① **w_tm_btn_L** 10 ns 펄스

② n_state 는 펄스와 동시에 SET, **c_state 는 다음 클럭** 에 SET

③ 펄스 동안 **o_tm_clear 1** → 다음 클럭 o_tm_run 0

④ 같은 클럭 59.94 → 00.00, o_tm_zero 1

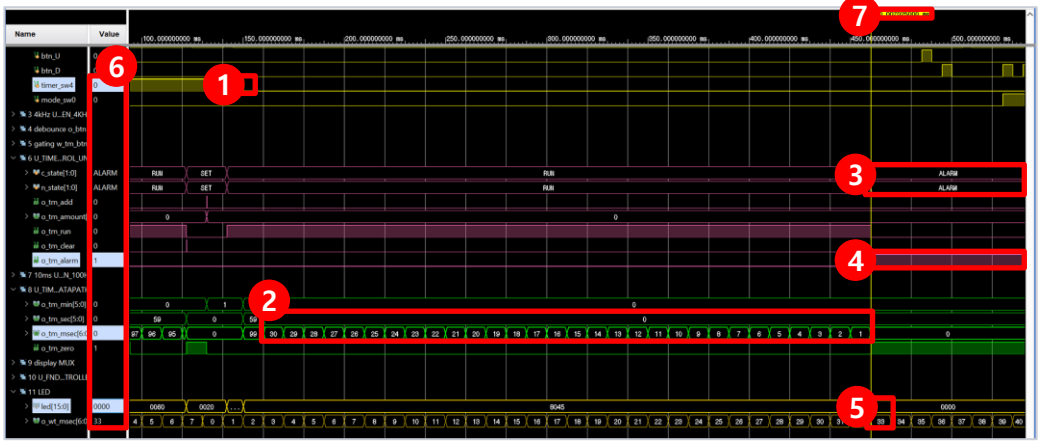
⑤ 커서 Value: L 1 · RUN / SET · clear 1

⑥ 커서 시각 122,004,005 ns

TM3 ⑤ P3 — 시계 화면에서도 타이머가 돌고 알람

150 ms 에 시계 화면으로 바꿔도 타이머는 계속 감 → 160 ms 프리로드 00:00.30 → 460 ms ALARM. LED 는 0000 (점멸의 꺼진 쪽)

TM3 · 93 ~ 535 ms · 커서 460.007025 ms (o_tm_alarm ↑) · 그룹 2 · 6 · 8 · 11



① 150 ms **timer_sw4 1 → 0** (시계 화면)

② 화면은 시계인데 타이머 값은 계속 감: 00:00.30 → 0

③ 460 ms **RUN → ALARM**

④ **o_tm_alarm 1** 유지

⑤ LED 8045 → **0000** — 시계 msec 33 < 50 이라 꺼진 쪽 (TB 도 0000 기대)

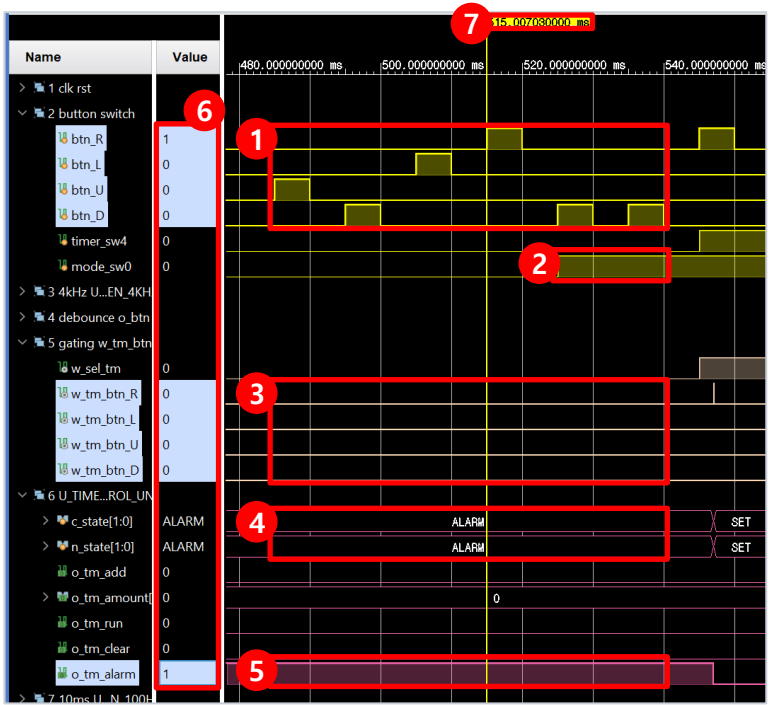
⑥ 커서 Value: sw4 0 · ALARM · alarm 1 · led 0000 · 33

⑦ 커서 시각 460.007025 ms

TM3 ⑥ P4 — 다른 화면 버튼은 알람을 못 끔

시계 화면 (485 ~ 525 ms) · 스톱워치 화면 (525 ms ~) 에서 U · D · L · R · D · D 를 눌러도 w_tm_btn_* 모두 0 → ALARM 유지

TM3 · 470 ~ 555 ms · 커서 515.007080 ms (btn_R 누름) · 그룹 2 · 5 · 6



① 시계 · 스톱워치 화면에서 U · D · L · R · D · D

② 525 ms **mode_sw0 1** → 스톱워치 화면

③ **w_tm_btn_* 4개 모두 0** (게이팅)

④ 상태 **ALARM 그대로**

⑤ **o_tm_alarm 1** 유지

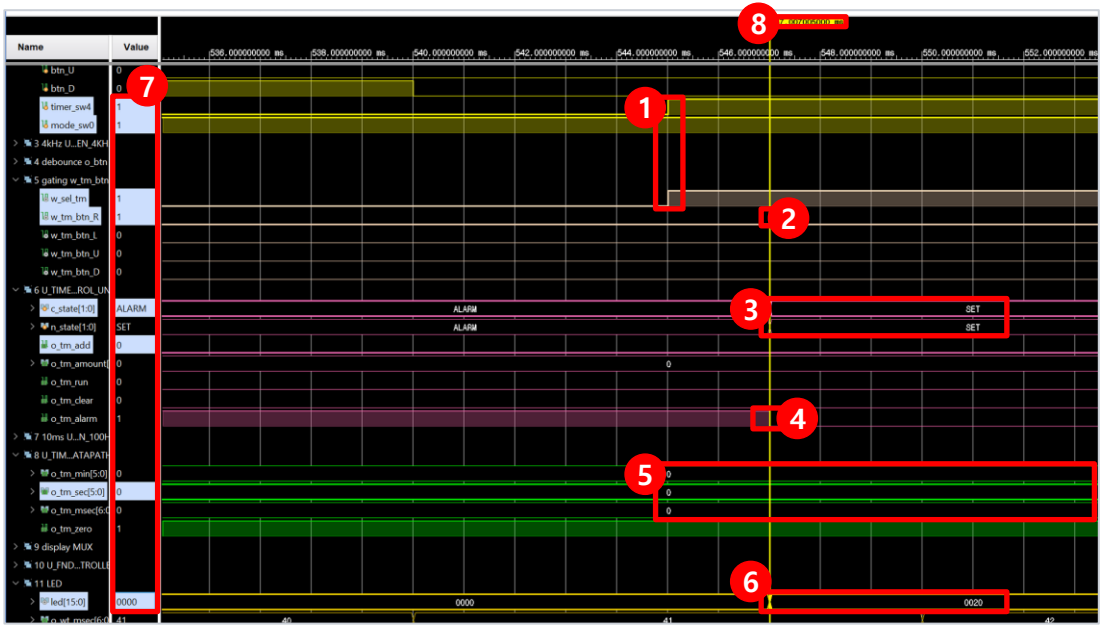
⑥ 커서 Value: btn_R 1 인데 w_tm_btn_R 0 · ALARM · alarm 1

⑦ 커서 시각 515.007080 ms

TM3 ⑦ P5 — 타이머 화면 우선 · R 로 알람 끄기

timer_sw4 · mode_sw0 둘 다 1 이면 타이머 화면 (w_sel_tm 1) — R 은 알람만 끄고 1분은 안 더함, LED 0020

TM3 · 535 ~ 554 ms · 커서 547.007005 ms · 그룹 2 · 5 · 6 · 8 · 11



① 545 ms **timer_sw4 1** (mode_sw0 도 1) → **w_sel_tm 1**

② R 펄스가 타이머로 (547.007 ms, 디바운스 뒤)

③ **ALARM → SET**

④ **o_tm_alarm 1 → 0**

⑤ 값 00:00.00 그대로 — 알람 중 R 은 끄기만

⑥ LED 0000 → **0020**

⑦ 커서 Value: sw4 1 · sw0 1 · sel_tm 1 · R 1 · ALARM/SET · add 0 · led 0000

⑧ 커서 시각 547.007005 ms

정리 · 팀이 알아야 할 것

타이머 사양 9가지를 TB 73개 검사 + 파형 24장으로 확인 — 기능 bug 0. 아래 3가지는 동작은 맞지만 설명이 필요한 부분

확인한 것

- 더하기 3종 (+10초 · +1분 · +10분) · 초→분 올림
- 59:59 상한 (U · R · D 모두)
- 00:00 에서 시작 안 함
- RUN 중 U · R · D 무시, L = 취소
- 실제 10초 카운트다운 · 분/초 빌림
- 알람 · LED 16개 500 ms 점멸
- 알람 중 아무 버튼 = 끄기 (값 안 바뀜)
- 백그라운드 (시계 화면에서도 알람)
- 화면 우선순위 · 버튼 게이팅

팀이 알아야 할 것

- ① **10초 → 9,998 ms** — 10 ms tick 을 셋이 공유해서, 시작 순간이 tick 사이 어디냐에 따라 첫 감소가 최대 10 ms 빨리 옴. TB 합격 범위 안. 시작마다 정확히 10 ms 로 하려면 타이머 전용 tick 필요 (10/16 PPA 측정 FF 약 +21개) — 팀 결정
- ② **알람 LED 시작 위치** — 점멸 박자가 시계 1/100초 기준이라, 알람이 울리는 순간 켜진 쪽 (ffff) 일지 꺼진 쪽 (0000) 일지는 그때 시계 값에 달림 (TM3 는 0000 부터)
- ③ **프리로드** — TM1 P6 · TM2 P4 · TM3 P3 는 TB 가 레지스터에 값을 직접 씀. 버튼으로 59:59 까지 올리는 경로는 TM1 P5 에서 실제 버튼으로 확인

범위: 시뮬레이션만 (보드 확인은 별도). 캡처 원본 · 상자판 · 배치 스크립트는 팀 저장소 boyun/timer_sim_verification